



# Práctica 6

## Organización y Arquitectura de Computadoras

Docente: Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

Tijuana, Baja California a 3 de octubre de 2025



## Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

## Dificultades durante el desarrollo

Durante el desarrollo de la práctica, la única dificultad que se presentó fue que no comprendía como se deben de acceder a cada elemento del 'arreglo', ya que, al momento de intentar invertir la cadena y capturarla se necesitaba tener la longitud previamente cargada en EDI, sin embargo, como se usaba el pushad y popad no comprendía él como podría obtener la longitud desde antes. Hasta que comprendí que la subrutina no solo era exclusiva para imprimir, si no que también podía hacer otras cosas cómo calcular la longitud.

## Desarrollo

1. (50%) Implementar una subrutina que reciba una cadena de texto direccionada por edx y la pase a mayúsculas, al final se debe mostrar en terminar con new\_puts.

El programa con una cadena ya capturada previamente guardada su dirección en EDX, entra en el bucle comparando primeramente si el carácter actual es el indicador de fin de carácter. (de ser así, se sale del bucle), posteriormente, se le resta '32' al carácter actual para pasarlo a su valor (en ASCII) mayúscula.

```
mayusculas:
    pushad
    mov edi, 0                ; se reinicia edi en cero
    change_mayus:
        cmp byte[ebx + edi], '%' ; se verifica si ya se llego al final de la cadena
        je fin_change          ; si se llega al final, se sale del bucle

        sub byte[ebx + edi], 32  ; se le restan 32 para que cualquier letra (a-z) se
convierta en mayuscula segun el ASCII
        inc edi                  ; se incrementa EDI para dirigirse al siguiente
caracter
        jmp change_mayus        ; se continua el bucle
    fin_change:
    popad
    ret
```

```
koria@Giolaptop:/mnt/c/Users/hugog/OneDrive/Documentos/6to Semestre/OAC/Practica6$ ./Practica6
Ingresa una cadena: (que termine en *):
comercio*
Cadena ingresada:
comercio
Cadena cambiada a mayusculas:
COMERCIO
```



## Código

```
%include "../Libreria/pc_io.inc" ; se incluye la libreria

section .text
    global _start:

_start:
    ; ===== CAPTURAR CADENA Y PASARLA A MAYUSCULAS
    ; imprimir prompt
    mov edx, prompt
    call newputs
    call salto

    mov ebx, cad ; se manda la direccion de la cadena a ebx
    call capturar ; se llama al procedimiento de capturar la cadena

    call salto
    mov edx, cad_ingresada ; se manda el contenido a ebx con el prompt para poder ser
    impreso
    call newputs ; se imprime el prompt de cadena capturada
    call salto

    ; imprime la cadena actualizada
    mov edx, cad
    call newputs
    call salto

    ; prompt cadena mayuscula
    mov edx, cad_mayus
    call newputs
    call salto

    ; conversion de la cadena en mayusculas
    mov ebx, cad
    call mayusculas

    ; impresion de la cadena en mayusculas
    mov edx, cad
    call newputs
    call salto

    ; ===== CAPTURAR CADENA E INVERTIRLA
    ; imprimir prompt
    mov edx, prompt2
    call newputs
    call salto

    ; se captura la cadena a invertir
    mov ebx, cad2
    call capturar
    call salto

    ; se imprime el prompt
    mov edx, cad_ingresada
    call newputs
    call salto

    ; se imprime la cadena ingresada para invertir
    mov edx, cad2
    call newputs
    call salto

    ; imprime prompt de cadena invertida
    mov edx, cad_inv
    call newputs
    call salto
    ; se invierte la cadena
```



```
mov ebx, cad2
call invertir

; se imprime la cadena ya invertida
mov edx, cad2
call newputs
call salto

; SYS_EXIT
mov eax, 1
mov ebx, 0
int 80h

; =====

capturar:
    pushad
    mov edi, 0
    ciclo_cap:
        call getche          ; se captura el caracter y se almacena en al
        cmp al, '*'          ; se compara con el indicador de fin de cadena
        je fin               ; si al == *, se sale del ciclo capturar

        cmp al, 13           ; si al == ENTER, no se captura
        je ciclo_cap

        cmp al, 10           ; si al == NUEVA_LINEA, no se captura
        je ciclo_cap

        mov [ebx + edi], al   ; se coloca el caracter capturado en ebx + indice (edi)
        inc edi               ; se incrementa el indice
        jmp ciclo_cap        ; se reinicia el ciclo

    ; imprimir cadena capturada
    fin:
        mov al, '%'          ; se coloca el caracter '%' en al
        mov [ebx + edi], al   ; se coloca el % al final de la cadena
    popad
    ret

mayusculas:
    pushad
    mov edi, 0                ; se reinicia edi en cero
    change_mayus:
        cmp byte[ebx + edi], '%' ; se verifica si ya se llevo al final de la cadena
        je fin_change         ; si se llega al final, se sale del bucle

        sub byte[ebx + edi], 32 ; se le restan 32 para que cualquier letra (a-z) se
                                ; convierta en mayuscula segun el ASCII
        inc edi                ; se incrementa EDI para dirigirse al siguiente caracter
        jmp change_mayus      ; se continua el bucle
    fin_change:
    popad
    ret

invertir:
    pushad
    mov esi, 0                ; se reinician nuestros registros de indices
    mov edi, 0                ; se reinician nuestros registros de indices

    calc_len:
        mov al, [ebx + edi]    ; se manda el contenido de la cadena a AL para ser comparado
        cmp al, '%'            ; se compara AL con el indicador final de cadena
        je fin_len             ; de ser cierto, se sale del bucle

        inc edi                ; se incrementa EDI para obtener la longitud de la cadena
        jmp calc_len           ; se reinicia el ciclo
    fin_len:
        dec edi                ; se decrementa EDI para no apuntar a '%'
    ciclo_invertir:
```



```
    cmp edi, 0          ; se compara EDI con 0 para ver si ya llegamos al principio
de la cadena          ; de ser cierto, se sale
    je fin_invertir

    mov al, [ebx + esi] ; se guarda el primer caracter en AL
    mov ah, [ebx + edi] ; se guarda el ultimo caracter en AH
    mov [ebx + esi], ah ; se hace el proceso de intercambio (ultimo -> primero)
    mov [ebx + edi], al ; se hace el proceso de intercambio (primero -> ultimo)

    cmp esi, edi        ; se compara ESI con EDI para ver si ya se llevo al centro
    JE fin_invertir     ; de ser cierto, se sale del bucle

    inc esi             ; se incrementa ESI para ser comparado
    cmp esi, edi        ; se compara ESI con EDI para ver si
    je fin_invertir     ; de ser cierto, se sale del bucle

    dec edi             ; se decrementa EDI para la siguiente iteracion
    jmp ciclo_invertir  ; se continua el bucle

fin_invertir:
popad
ret

newputs:
pushad
mov esi, 0
prnt:
    mov al, [edx + esi] ; se manda a al el caracter inicial
    cmp al, '%'         ; se compara al con %
    je finputs          ; si es verdadero, se sale del ciclo

    call putchar        ; se imprime al caracter actual
    inc esi             ; se incrementa el indice
    jmp prnt            ; se reinicia el ciclo de impresion

finputs:
popad
ret

salto:
pushad
mov al, 13
call putchar

mov al, 10
call putchar
popad
ret

section data
prompt: db "Ingresa una cadena: (que termine en *):%"
prompt2: db "Ingresa una cadena a invertir (que termine en *):%"

cad_ingresada: db "Cadena ingresada:%"
cad_mayus: db "Cadena cambiada a mayusculas:%"
cad_inv: db "Cadena invertida:%"
vocales: db "Contador de vocales:%"
cantVocales: db "Total de vocales:%"
VocalesA: db "Total de A:%"
VocalesE: db "Total de E:%"
VocalesI: db "Total de I:%"
VocalesO: db "Total de O:%"
VocalesU: db "Total de U:%"
```



```
section .bss
    cad resb 256
    cad2 resb 256

    contA resb 1
    contE resb 1
    contI resb 1
    contO resb 1
    contU resb 1
    suma resb 1

    cad_len resb 1
```

## Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)